

Exercise Specification

184.153 Distributed Systems Engineering

2026S, VU, 2.0h, 3.0EC

Vienna University of Technology

Autor	Univ.-Lektor ZT Dipl.-Ing. Dr. techn. Johannes Weidl-Rektenwald Staatlich befugter und beeideter Ziviltechniker für Informatik
Datum	09.03.2026
Version	1.1
Status	Final
Copyright	This exercise specification must not be used , in whole or in part, without the author's written consent for any purpose other than the participation and completion of this exercise. All rights reserved.
Contact	J. Weidl-Rektenwald: jwr@dsg.tuwien.ac.at DSE Team: dse@dsg.tuwien.ac.at

Change History

VERSION	DATE	STATUS/CHANGES	AUTHOR
0.1	05.01.2026	Draft	JWR
0.2	25.02.2026	Draft	JWR
0.3	28.02.2026	Draft	JWR
0.4	01.03.2026	Draft	JWR
1.0	02.03.2026	Final	JWR
1.1	09.03.2026	Final: Sharpening requirements, corrected typos	JWR

Table of Contents

1	Introduction.....	4
2	Exercise specification	7
2.1	Functional requirements	8
2.2	Requirements for services and interfaces	9
2.3	Requirements for the solution architecture.....	10
1.1.1	Sketching the logical software architecture.....	11
2.4	Deployment requirements	12
1.1.2	Local operation of Kubernetes	12
2.5	Requirements for technical implementation	13
2.6	Requirements for the simulation scenario	15
2.7	Requirements for the submission interview	19
2.8	Code submission requirements.....	19
2.9	Testing requirements.....	19
3	Deliverables	19
3.1	Project plan.....	19
3.2	Architecture and Design Document	20
3.3	Implementation	20
3.4	Maintenance Manual	21
3.5	Project Closure Report.....	21

4	Evaluation of the deliverables.....	21
5	Exercise procedure and organization.....	23
5.1	Important notes and tips.....	24
5.2	Important dates.....	25
6	Summary of the exercise contents.....	25

Table of Figures

Figure 1: Sketch of the logical software architecture	11
Figure 2: Simulation Scenario 2 (schematic)	18

1 Introduction

In the last 10-15 years we saw many newcomers entering the field of the big classic Automotive Tier1 supplier companies and OEMs (Open Equipment Manufacturers) presenting disruptive new electronics and software architecture models redefining the way of how vehicles are controlled via electronic control units (ECUs) - e.g. Tesla, Rivian, and BYD just to mention three. Goal of those disruptive vehicle platforms is to reduce the number of ECUs in a vehicle together with the wiring requirements and producing a so-called “software defined vehicle (SDV)”, where software rather than hardware and electronics define the major functions of the vehicle in a flexible, over-the-air updateable and pay-per-use-ready model.

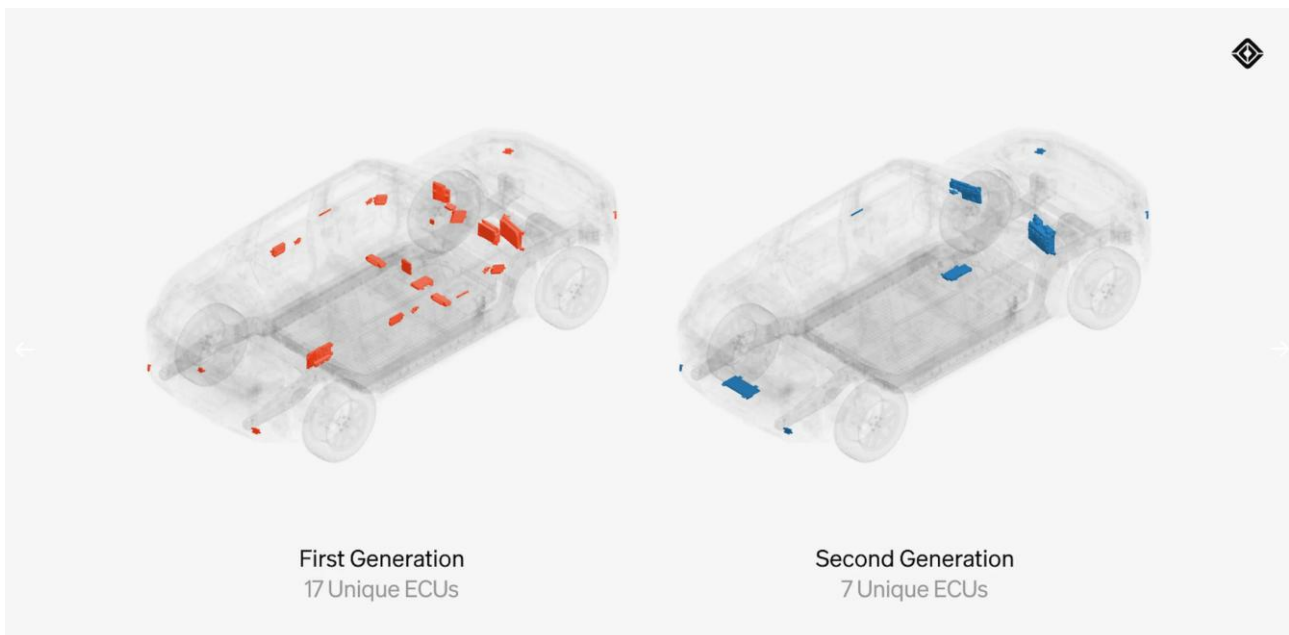
Still most vehicles on European roads are controlled by a high number of single interconnected ECUs, 40 to 150 (!) per vehicle. Whereas this architecture model has advantages in being able to source many different ECU suppliers and building up a vehicle in a kind of LEGO® model, it has vast disadvantages in hardware and software duplication tied to high costs of purchasing CPUs and hardware, wiring and software stacks. In addition, as weight is a paramount factor in the production of vehicles, reducing weight is another benefit of reducing the number of ECUs and the wiring in between, especially for Electronic Vehicles (EVs), where weight has a direct correlation to battery performance. Furthermore, the vision of a software-defined vehicle (SDV) shall be supported by such new electronics architecture, allowing a vast number of flexible software features and options by simply adapting the software on a standard and high-performance hardware infrastructure.

Some OEMs tried to develop such disruptive ECU architecture models themselves, centralizing originally distributed software departments into central software companies with the clear target to produce such new architecture models in 3 to 5 years' time. However, some of those OEMs – for whatever reason ever – failed achieving their vision.

This for example gets evident in the investment of the Volkswagen Group into Rivian, a US startup company, producing cars themselves, but started on the green field developing a completely new ECU architecture [see <https://www.volkswagen-group.com/de/pressemitteilungen/schneller-schlanker-effizienter-rivian-und-volkswagen-group-starten-joint-venture-18828>]. Before investing into a joint venture with Rivian, Volkswagen group tried to develop their own new architecture model in the centralized software company CARIAD (“Car I Am Digital”).

The Joint Venture's goal is to adapt Rivian's greenfield-developed ECU software platform to integrate it into the historically grown ecosystem of Volkswagen, powering 12 passenger vehicle brands, hundreds if not thousands of car models and – an estimated - hundred thousand different car configurations.

Rivian – founded in 2009 – started with a domain-based ECU architecture but switched, in its second-generation product line, to a ‘zonal architecture’. This allowed Rivian to reduce the number of ECUs from 17 to 7 [see <https://www.popsci.com/technology/rivian-zonal-electrical-architecture/>]:

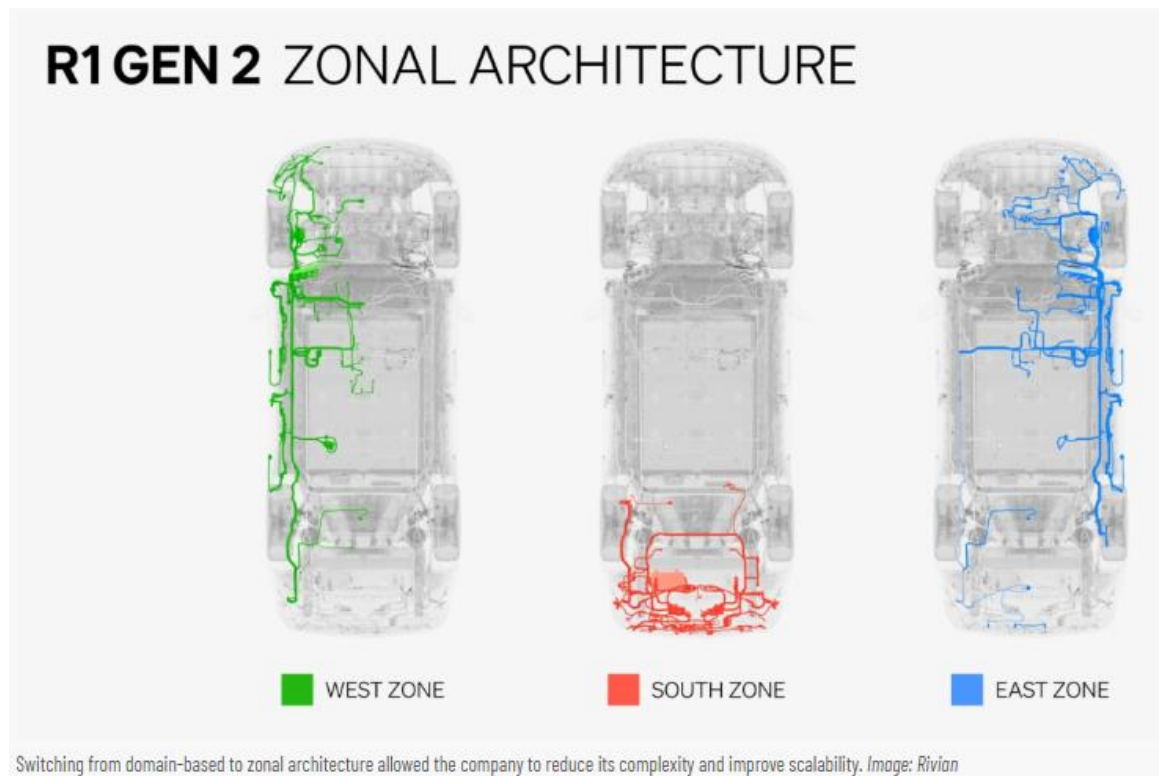


Again, advantages of this reduction in the number of ECUs are reduced weight, reduced wiring, reduced hardware and software diversity and costs, reduced opportunities for failure, reduced integration time into the vehicle, reduced construction space, reduced complexity/time of software updates.

Rivian mentions in the article <https://www.popsci.com/technology/rivian-zonal-electrical-architecture/>:

“For the second generation of vehicles, four categories get their own ECUs: infotainment, autonomy, vehicle access, drive units, and its battery management system. Every other vehicle function is controlled by just three ECUs Rivian refers to as West, East, and South. The infotainment ECU alone is as powerful as a laptop and has the capabilities of a smartphone.”

The following picture describes the zonal architecture:



Rivian also developed a platform for autonomous driving, the Rivian Autonomy Platform. It is using ultrasonic sensors, cameras and radars to enable the detection of the vehicle surroundings [see <https://stories.rivian.com/meet-the-new-r1>].

Rivian includes NVIDIA DRIVE Orin processors and the NVIDIA DRIVE OS to power the Rivian Autonomy Platform, and Rivian refers to utilizing AI software to anticipate, plan and react in real time.

Rivian – by using NVIDIA DRIVE OS - is utilizing one of the current market leaders in autonomous driving software, together with Tesla, Waymo, Cruise, Mobileye, and Baidu being the main market competitors.

Seeing a lot of modern and disruptive development in the automotive software industry in the last decade - including using Cloud backends for enabling numerous use cases – and microservices being around also for about a decade, I did not yet see the concept of microservices directly executed on ECUs by utilizing e.g. orchestration frameworks such as Kubernetes. Of course, the automotive industry must always consider their high targets in achieving functional safety up to ASIL Level D (see also the standard for automotive functional safety ISO 26262).

Nevertheless, this is interesting, because using microservices and orchestration frameworks in vehicles would provide – beneath others - the following advantages:

1. Microservices could be migrated between different ECUs yielding location transparency, fail-operational computation, and autoscaling.
2. Microservices could be migrated between the vehicle and the (Cloud) backend yielding location transparency, fail-operational computation, autoscaling and interruption-free computing while the car is powered down.
3. Microservices could support a clean software architecture supporting concepts and patterns such as the Single Responsibility Principle (SRP), Separation of Concern (SoC), high Cohesion and low Coupling, and thereby a smooth Software Evolution and the capability of fast service innovation and reduced maintenance costs.

A lot of use cases can be conceived using a Microservices Architecture (MSA) being used inside a vehicle. One of this use cases we will implement in this exercise, trying to set grounds for an even more advanced automotive software architecture powering Software-Defined Vehicles in the future.

2 Exercise specification

In this exercise, you will implement an innovative mixed ECU/backend solution for Vehicle-to-Infrastructure (V2I) communication in the field of assisted driving.

All the software modules - running on the ECU and/or in the backend - shall be implemented as “microservices”.

In contrast to a monolithic system, this results in a highly distributed system, which means that fundamental problems of distributed systems in design, implementation, deployment and operation must be solved.

The "orchestration" of the microservices shall be carried out via "Kubernetes" [<https://kubernetes.io/>] to be able to support challenges such as automated deployment, auto-scaling and fault tolerance as automatically as possible.

A user interface to inspect the status of a specific scenario to be implemented shall be designed as a web application.

2.1 Functional requirements

In the exercise, the following use case shall be supported:

- The hypothetical OEM “*DRIVE2STARS (D2S)*” is working on a vehicle ECU platform software which runs vehicle functions as microservices on the Kubernetes orchestration platform.
- “*D2S*” vehicles are using this ECU platform software to drive on public streets, leveraging assisted and automated driving functionalities provided by microservices hosted on the ECU platform software.
- The following microservices shall be used in the exercise ecosystem – hosted either on the beforementioned vehicle ECU software platform, the Kubernetes-based OEM backend, or in any other Kubernetes hosted infrastructure:
 - **WHEREAMI Microservice:** The WHEREAMI microservice reads the vehicle’s GPS location and sends it periodically to any configured endpoint.
 - Endpoints can be: Another service in the vehicle itself, in the OEM backend, in other vehicles, in backends of authorities, backends of other vendors etc.
 - **UTRACKED Microservice:** The UTRACKED microservice collects data from the WHEREAMI microservices and stores this data in a database infrastructure to keep track of the movement history.
 - **SONAR Microservice:** The SONAR microservice monitors the distance to a vehicle in the front or in the back using fused data out of radar, LIDAR and ultrasonic sensor data to optimize sensor data quality. The distance is measured in meters. Additionally, the distance change in meters/second is computed. Both values are periodically sent to any configured endpoints.
 - **BRAKENOW Microservice:** The BRAKENOW emergency brake microservice - based on certain conditions - triggers the vehicle’s braking system to put the vehicle to an emergency stop in case of any danger for the vehicle’s integrity of the life of its passengers detected.
 - Those conditions are:
 - a distance < 30 meters and a distance-decrease of > 4 m/s (decreasing the distance between the vehicles)
 - a distance < 15 meters and a distance-decrease of > 2 m/s (decreasing the distance between the vehicles)
 - a distance < 5 meters and a distance-decrease of > 0 m/s
 - In case the distance to a vehicle in the front/back is < 45 meters, the so called “pre-emergency brake state” is entered. The state list left when the distance of vehicles in the front/back is >= 45 meters.
 - **ORCHESTRATOR Microservice:** The ORCHESTRATOR microservice works on the data collected by different other services and triggers other services to execute certain actions based on the input data and certain rules.
- The ORCHESTRATOR service shall use UTRACKED service data to compute the distance between all data sending vehicles.

- The ORCHESTRATOR service shall receive data of the SONAR service and shall check vehicle distance data computed from the UTRACKED service and received from the SONAR service continuously for plausibility.
 - In case the SONAR service identifies
 1. a distance < 30 meters and a distance-decrease of > 4 m/s (decreasing the distance between the vehicles)
 2. a distance < 15 meters and a distance-decrease of > 2 m/s (decreasing the distance between the vehicles)
 3. a distance < 5 meters and a distance of > 0 m/s
 4. a deviation of > 15 meters of data in UTRACKED and SONAR
 - the ORCHESTRATOR service shall send the BRAKENOW service the signal to execute an emergency brake.
 - The first three conditions are evaluated in parallel to the BRAKENOW service to achieve redundancy in the condition evaluation and triggering for an emergency brake.
 - The fourth condition is the value add of the ORCHESTRATOR service.
 - The BRAKENOW service shall inform the ORCHESTRATOR if a pre-emergency-brake state or an emergency brake has been triggered locally. The ORCHESTRATOR persists this data in its database. The ORCHESTRATOR also persists the event of sending an emergency brake signal to the BRAKENOW service.

Because of the location transparency of the specified microservices, the microservices can be hosted on any ECU in the vehicle platform software as well as in any Kubernetes-driven backend. Furthermore, microservices can be transferred flexibly between the infrastructures. Microservices can also be provisioned to run more than one instance at a time to achieve scalability and fault tolerance.

A web application, the so called “**Storefront**”

1. shall display vehicles in a map view, where the position is continuously updated.
 - a. The type of map view (Google Maps, OpenStreetMap, other map view, schematic) is a free design decision in the implementation.
2. shall show all vehicles in an identifiable way (possibly by a short form of the vehicle identification number or VIN – see https://en.wikipedia.org/wiki/Vehicle_identification_number).
3. shall display in an event log view all relevant historical data, newest events first in the list, timestamps in the first column.

To show an example scenario, a simulation scenario shall be implemented accordingly.

2.2 Requirements for services and interfaces

1. Human users operate the system via a web application in a browser.
2. Vehicles communicate with the backend via a message broker or an API gateway.

3. For each microservice, a suitable interface must be designed regarding optimal design in terms of coupling and cohesion or "bounded context" (see the topics of the lecture).
4. The microservices are each to run in a container to take advantage of containerization.
5. A suitable data storage system must be designed for each microservice. It is **not** allowed for microservices to access the data storage of other microservices directly (i.e. bypassing the "public API" of the partner microservice).

2.3 Requirements for the solution architecture

The basic solution architecture consists of the following basic building blocks:

- 1) Spider (API Gateway)**
 - a. Spider shall act as API gateway for the different clients.
 - b. The API gateway should have a REST, GraphQL or gRPC interface.
- 2) MoM (Message Broker)**
 - a. For asynchronous communication with the backend, a message broker is used.
- 3) Storefront**
 - a. The Storefront implements a web-based user interface for the visualization/operation of the required simulation scenario.
- 4) WHEREAMI**
 - a. The WHEREAMI microservice runs once per vehicle and sends data to the backend.
- 5) UTRACKED**
 - a. The UTRACKED stores the GPS data for the different vehicles which are identified by their vehicle identification number (VIN).
- 6) SONAR**
 - a. The SONAR microservice runs once per vehicle and continuously computes the distance to the vehicle in front and in the back plus the distance change rate.
- 7) BRAKENOW**
 - a. The BRAKENOW microservice - based on the defined conditions - triggers the vehicle's braking system.
- 8) ORCHESTRATOR**
 - a. The ORCHESTRATOR service implements a control instance for vehicles, i.e. it can send data to vehicles, e.g. to perform assisted and automated driving functions.
- 9) Database**
 - a. Databases shall store non-volatile data. Databases used do not have to run in Kubernetes, but cross-access by one microservice into the data storage of another microservice is strictly prohibited.
- 10) Simulator**
 - a. In the absence of real vehicles, the Simulator service provides the data that comes usually from the vehicle sensors, e.g. GPS location and distance to other vehicles. The simulator simulates sensors which can send their sensor data locally to the SONAR service, but also directly to the backend where it is received by a SONAR service running on the backend for further processing.

1.1.1 Sketching the logical software architecture

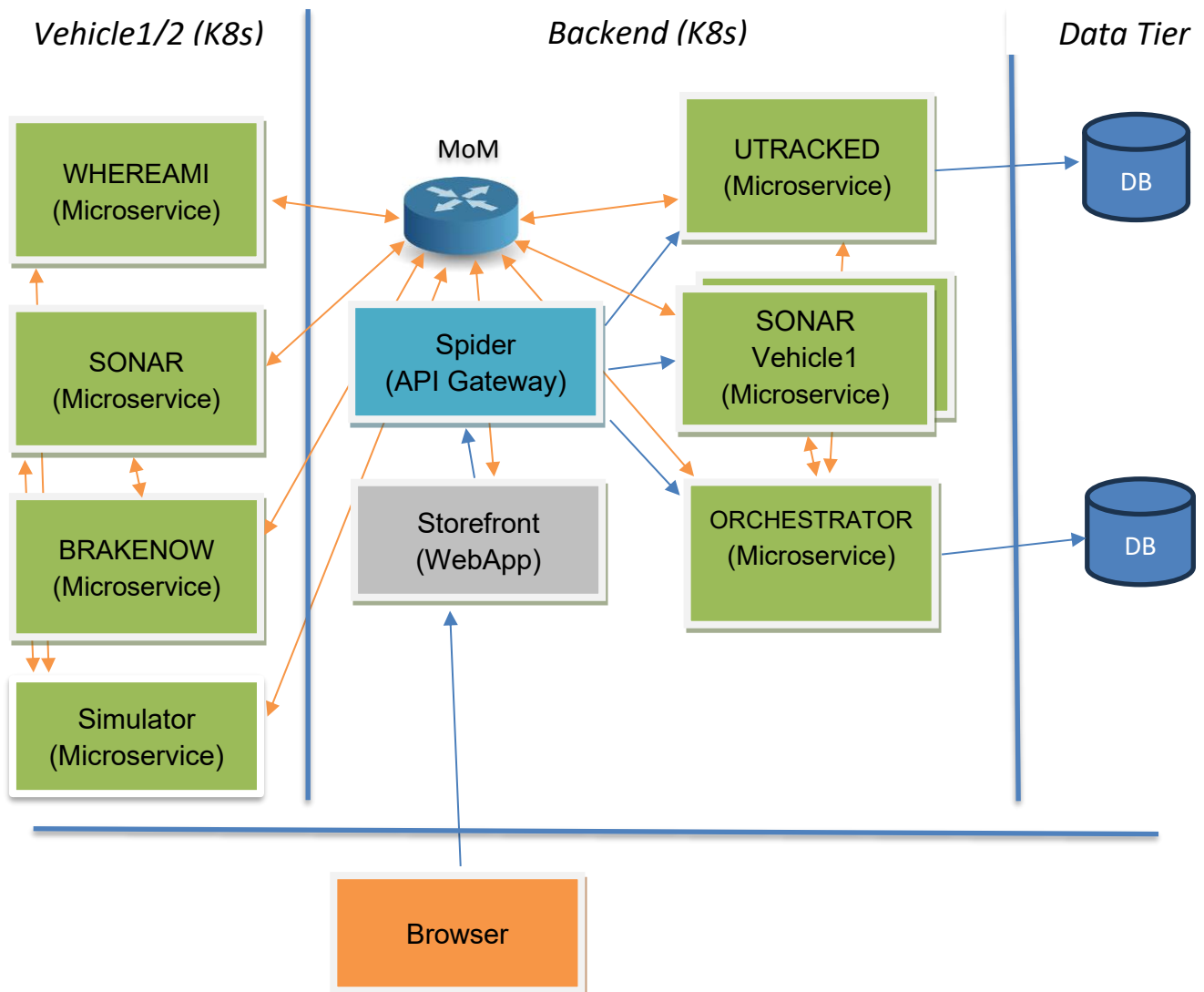


Figure 1: Sketch of the logical software architecture

- Each microservice that needs to store data permanently must have its own/separate data storage! A microservice is therefore not allowed to access the "database" of another microservice directly!
- The microservices can communicate directly with each other, either synchronously via API calls or asynchronously via the message broker.
- The microservice running on the vehicle and on the backend shall use exactly the same source code. Any differences in behaviour shall be only addressed via configuration.

2.4 Deployment requirements

This exercise uses Google Cloud Platform™ (GCP). After the end of the group registration, a coupon will be provided per group member (see also information on TUWEL). Google® is thankfully supporting this exercise with a free budget for the use of GCP.

ATTENTION:

1. Be sure to redeem your credit immediately after receiving the coupon, as there is an expiration date for redemption. **Not redeemed coupons expire completely, the redemption period will not be extended by Google!**
2. See also the following guides: [how to redeem education grants](#), [cost management tips](#), and [billing and team projects](#) (to use more coupons on a single group project).
3. Information about redeeming and using the coupons will also be available on TUWEL.

Deployment Requirements:

1. Container
 - a. Each microservice is to run in its own **container**.
2. Orchestration
 - a. To orchestrate the containers, use **Google Kubernetes Engine (GKE)** and create a corresponding Kubernetes cluster.
 - b. The cluster **must** be created in Standard cluster mode, not Autopilot mode.
3. Database
 - a. Since Docker containers are generally *stateless* - which does not necessarily meet the goal of persistent data storage - a Google database service can be used to store the data, i.e. data storage does not have to be implemented within a microservice.
 - b. The microservices in the containers then access demarcated areas in this database service – as already mentioned, there must be no cross-access by microservices to demarcated data areas that do not belong to the corresponding microservice.
4. Messaging
 - a. The message broker (e.g. RabbitMQ, but the selection is a free design decision) can run in its own container or Google services (e.g. Cloud Pub/Sub) can be used to implement message-based communication.

The code submission must be completely deployable in the sense of IaC (Infrastructure as Code), i.e. you must submit your Kubernetes manifests (YAML files for deployments, services, etc.) deployable.

1.1.2 Local operation of Kubernetes

The following frameworks are suitable for – optional – local development based on Kubernetes:

1. Minikube (<https://github.com/kubernetes/minikube>)
2. microk8s (<https://microk8s.io/>)

3. Docker-for-Mac (<https://docs.docker.com/docker-for-mac/kubernetes/>)
4. Docker-for-Windows (<https://docs.docker.com/docker-for-windows/kubernetes/>)
 - a. Attention: The use of Docker-for-Windows causes problems, as the installation automatically activates Hyper-V and thus VirtualBox no longer works. So if you need VirtualBox, Docker-for-Windows is unfortunately not an option.

GCP/Kubernetes/StatefulSet Tutorials:

1. <https://cloud.google.com/kubernetes-engine/docs/>
2. <https://cloud.google.com/kubernetes-engine/docs/tutorials/hello-app>
3. <https://cloud.google.com/kubernetes-engine/docs/tutorials/guestbook>
4. <https://cloud.google.com/kubernetes-engine/docs/tutorials/persistent-disk>
5. <https://cloud.google.com/kubernetes-engine/docs/concepts/statefulset>

2.5 Requirements for technical implementation

The following must be used:

1. Kubernetes (Google Kubernetes Engine on Google Cloud Platform)
2. A container technology (e.g. Docker)
3. REST/GraphQL/gRPC interface for the API gateway

The following frameworks can be used, but are not required:

1. Spring Boot [<http://projects.spring.io/spring-boot>]
 - a. Spring Boot is particularly suitable for implementing microservices.
2. DropWizard [<http://dropwizard.io>]
 - a. DropWizard is particularly suitable for the implementation of microservices.
3. Quarkus [<https://quarkus.io/>]
4. Circuit Breaker Hystrix [<https://github.com/Netflix/Hystrix>]
5. Mongo DB with Spring Data MongoDB [<http://projects.spring.io/spring-data-mongodb/>]
 - a. Spring Data MongoDB provides a Java repository abstraction to access MongoDB functions.
6. Service Mesh Istio [<https://istio.io>]
7. Service Mesh linkerd [<http://linkerd.io>]
8. resilience4j [<https://github.com/resilience4j/resilience4j>]

More tutorials and resources:

1. Docker Tutorial
 - a. <https://www.docker.com/tryit/>
2. Hystrix Tutorial

- a. <https://github.com/Netflix/Hystrix/wiki/How-To-Use>
3. Spring Boot Tutorial
 - a. <https://spring.io/guides/gs/spring-boot/>
4. MongoDB Geospatial Queries:
 - a. <https://docs.mongodb.com/manual/geospatial-queries/>
5. Building Microservices with Spring Boot
 - a. <http://www.infoq.com/articles/boot-microservices>
 - b. <https://blog.scottlogic.com/2019/10/31/building-microservices-with-spring-boot.html>
6. GKE – Google Kubernetes Engine
 - a. <https://cloud.google.com/kubernetes-engine/docs/>
7. Docker Get Started
 - a. <https://docs.docker.com/get-started/>
8. Docker Overview
 - a. <https://docs.docker.com/engine/docker-overview/>
9. Kubernetes „Hello World“ Example
 - a. <https://cloud.google.com/kubernetes-engine/docs/tutorials/hello-app>
10. MongoDB on GCP
 - a. <https://codelabs.developers.google.com/codelabs/cloud-mongodb-statefulset/>
 - b. <https://cloud.google.com/solutions/deploy-mongodb?hl=de>
11. k8s – OpenShift - GKE comparison
 - a. <https://www.slideshare.net/deview/d2-community-open-container-seoul-meetup-kubernetes-openshift>
12. OpenShift / Kubernetes difference
 - a. <https://www.redhat.com/de/blog/openshift-and-kubernetes-whats-difference>

Google Cloud resources:

1. Watch the [tour of the Cloud Console](#).
2. For general Cloud training, check out [suggested learning paths](#), [quickstarts](#), [sample projects](#), and the [Google Cloud Documentation](#) site.
3. Watch the [Google Cloud Platform Essentials video series](#) on YouTube.

2.6 Requirements for the simulation scenario

Create a simulator to demonstrate the following three (3) scenarios:

General requirements:

1. All the important information shall be shown in the Storefront web application
 - a. Road (schematic)
 - b. First vehicle
 - c. Second vehicle
 - d. Info from the microservices in both vehicles (e.g. in separate window parts)
 - i. GPS location of the vehicle
 - ii. Speed
 - iii. Distance to front/rear vehicle
 - iv. Distance change per second to front/rear vehicle
 - v. Emergency brake active (true/false)
 - e. Info from the microservices running on the backend (e.g. in another separate window part)
 - i. GPS locations of the vehicle
 - ii. Distance to front/rear vehicle per vehicle
 - iii. Event log from the ORCHESTRATOR, e.g. you can see when the ORCHESTRATOR sends the emergency brake command to the Emergency Brake Microservice on vehicle 2.
2. Show the scenarios described with meaningful parameters assumed in real time in the Storefront.

Scenario 1:

3. Two vehicles drive on a long straight road (or on a circuit)
4. Both vehicles drive at the same speed.
5. Information in (near) real-time is visible, no emergency brake is triggered in this scenario.

Scenario 2:

1. Two vehicles drive on a long straight road (or on a circuit)
2. The first vehicle drives at a constant speed v_1 .
3. The second vehicle drives at a faster speed v_2 than the first vehicle ($v_2 > v_1$)
4. The distance from the second to the first vehicle hence continuously decreases
5. The prerequisite for the “pre-emergency brake state” shall be fulfilled at a certain point in time and visualized in the storefront (be bold here, we do not want to see vehicles crash and shall depict the danger accordingly in the storefront GUI)

6. The prerequisite for an emergency brake shall be fulfilled at a certain point in time
7. As soon as the conditions for an emergency brake are fulfilled, the second vehicle immediately stops due to the activation of the emergency braking system (you can assume the vehicle stops abruptly, without having to simulate the braking path and decrease in speed).
8. The first vehicle continues driving, the second vehicle keeps standstill
9. As soon as the distance between the vehicles is $> 100\text{-}200$ meters (you can choose a reasonable value in this range), the second vehicle regains its original speed (by doing so, the scenario repeats indefinitely, some iterations shall be shown in the demonstration)
10. This scenario is also depicted graphically in Figure 2.

Scenario 3:

1. Two vehicles drive on a long straight road (or on a circuit)
2. The first vehicle drives at a constant speed v_1 .
3. The second vehicle drives at a faster speed v_2 than the first vehicle ($v_2 > v_1$)
4. The distance from the second to the first vehicle hence continuously decreases
5. The prerequisite for the “pre-emergency brake state” shall be fulfilled at a certain point in time and visualized in the storefront
6. Vehicle 1 increases its speed with $v_1 \gg v_2$
7. The prerequisite for the “pre-emergency brake state” vanishes – this shall be visualized in the storefront GUI

Optional: For a better simulation of the scenario on a controllable timeline, the flux compensator technology described by the author can be used – see <http://coopxarch.blogspot.co.at/2011/05/der-fluxkompensator-und-seine-bedeutung.html>

Optional: Show the process in timelapse (e.g. factor 2-10).

Optional: Install the service mesh 'linkerd' and show how to display data collected by linkerd regarding microservice communication.

Optional: Show how microservices can be activated/deactivated on the vehicle and/or on the backend while the basic functionality of the simulation scenario keeps intact. Render buttons on the GUI to activate/deactivate those microservices during the scenario is executed.

Optional: Show how fault tolerance can be implemented, both on vehicles, on the backend, and across vehicles and the backend. For example: When the SONAR microservice is deactivated on a vehicle, the SONAR service on the backend still is active and triggers the emergency brake service, even the function is currently not available/running on one of the vehicle's ECUs.

Optional: Show the functionality of an in-drive ECU update. Step 1: All services of the ECU to be updated are moved to another ECU or to the backend. Step 2: The ECU is updated (OS version, Kubernetes version, or whatever). Step 3: The services previously moved away from the ECU that was

subject to software update are moved back to the ECU. This scenario shows one of the big advantages of our microservice-based vehicle ECU platform. Even if the ECU is capable of A/B (i.e. seamless) update capabilities, a reboot of the ECU is still needed, which usually cannot be done while driving. Note that the functional safety aspect in terms of the [ISO 26262](#) of such in-drive software update clearly must be addressed.

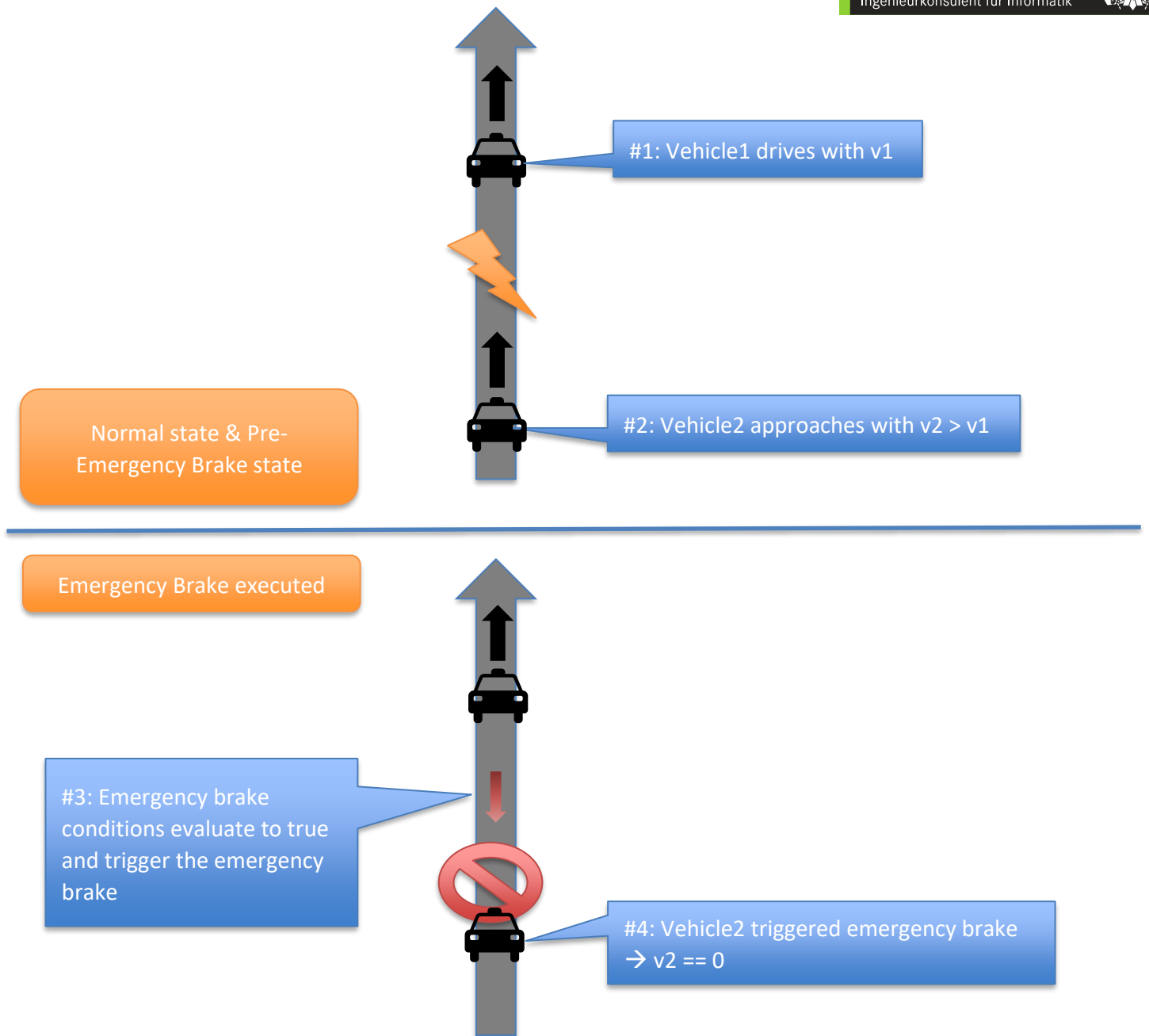


Figure 2: Simulation Scenario 2 (schematic)

2.7 Requirements for the submission interview

In the submission interview, you will present the structure and function of your solution based on the live implementation of the simulation scenario hosted in the Google Kubernetes Engine environment. Further details on the submission interviews are presented in Chapter 5.

2.8 Code submission requirements

The code submission must be completely deployable in the sense of IaC (Infrastructure as Code), i.e. you must submit your Kubernetes manifests (YAML files for deployments, services, etc.) deployable.

2.9 Testing requirements

For a selected microservice, at least two (2) "module tests" must be written that test the functionality via the microservice API. The use of a test framework is optional.

3 Deliverables

The documents supplied can be written either in German or English. Source code documentation and API documentation must be written in English. All deliverables must be created and submitted - there are no optional deliverables.

3.1 Project plan

Create a document with the following content:

Deliverable 1: Document: Project plan

1. Project Goal Plan: Contains a short project description with the project goals and non-goals
2. Project Organization Chart: Contains the assignment of roles in the project team
 - a. Who takes on which role or tasks? Who has what responsibility?
3. Work Breakdown Structure (WBS): The WBS contains the work package specifications
 - a. What are the individual tasks that must be solved within the exercise?
4. Effort estimation: What effort do you plan for the activities of the WBS?
 - a. What is the estimated effort for each project team member (in person-hours)?
 - b. What is the estimated total effort (in person-hours)?
5. Resource planning and milestone planning
 - a. Who in the team completes which work package by when?

- b. Determination of completion dates for the work packages and deliverables
6. Engineering planning: What methods, technologies and tools will you use and for what purpose?
7. GCP Budget Estimate: Estimate how much GCP budget you will need to complete the exercise on Google Cloud Platform. Evaluate the GCP cost model – there may be services that are offered free of charge up to a certain level of use.
8. *Optional*: Project Gantt chart (classic) or sprint planning (agile)

Please note that the project plan must be submitted **on time**!

3.2 Architecture and Design Document

Create a document with the following content:

Deliverable 2: Document: Architecture and Design Document

1. Description of the software architecture
 - a. Which components / microservices exist?
 - b. Which interfaces exist?
 - c. How do the services communicate with each other?
2. Display the "Logical View" and the "Physical or Deployment View" as in the "4+1 Architectural Model" by Philippe Kruchten - see also
[\[http://en.wikipedia.org/wiki/4%2B1_Architectural_View_Model\]](http://en.wikipedia.org/wiki/4%2B1_Architectural_View_Model)

3.3 Implementation

Implement the distributed system described in the Exercise Specification.

Deliverable 3: Code submission: Implementation of the requested requirements/modules

3.4 Maintenance Manual

Create a document with the following content:

Deliverable 4: Document: Maintenance Manual

1. Description of the development environment
2. Description of the frameworks and libraries used
3. Description of the build process
4. Description of the testing process
5. Description of the deployment
6. Documentation of the API Gateway API using Swagger (<https://swagger.io>) or a comparable API documentation framework

3.5 Project Closure Report

Create a project closure report including lessons learned and a conclusion.

Deliverable 5: Document: Project Closure Report

1. Assess the progress of the project
 - a. What worked well, what didn't work well? What are the lessons learned? Justify your assessment.
 - b. If applicable, why did you exceed the initial estimated effort?
 - c. What would you do differently or better based on the experience you have gained?
2. Assess the final software
 - a. What went well, which parts would you solve differently/better next time.
3. Write your personal conclusion about the course.

4 Evaluation of the deliverables

Deliverable 1: Document: Project plan

5 points – Deductions in case of incompleteness or lack of plausibility:

1. **0.5 points** - Contains a short project description with the project goals and non-goals (project goal plan)
2. **0.5 points** - Includes the assignment of roles in the project team (project organizational chart)
 - a. Who takes on which role or tasks? Who has what responsibility?
3. **1 point** - Contains the Work Package Specification or Work Breakdown Structure (WBS)
4. **1 point** – Includes the effort estimation
5. **1 point** - Includes Resource Planning and Milestone Planning

- a. Who in the team does what and by when?
- b. Determination of completion dates for the work packages and deliverables
- 6. **1 point** – Includes the engineering and GCP budget estimate
 - a. What methods and tools will you use and why?
 - b. The technical planning is described in detail.
 - c. How much GCP budget do you plan to consume?

Deliverable 2: Document: Architecture- and Design Document

5 points – Deductions for incompleteness or incomprehensible design decisions that are in contrast to the state-of-the-art taught in the lecture.

Deliverable 3: Code submission: Code submission: Implementation of the requested modules

45 points – Deductions for incompleteness: missing API documentation, missing inline comments, missing functionality, lack of communication, lack of independence of the components, non-functioning user interface, unauthorized trivialization, lack of tests, lack of use of prescribed frameworks.

Deliverable 4: Document: Maintenance manual

3 points – Deductions for incompleteness compared to the task.

Deliverable 5: Document: Project Closure Report

2 points – Deductions for incompleteness compared to the task.

Maximum score: **60 points**

Minimum score: **30 points**

Note: No points are awarded for the implementation of optional requirements (marked with the word "Optional:"). This means that no points for options can be "exchanged" for points of mandatory parts that have not been achieved.

5 Exercise procedure and organization

1. Exercise groups of three (3) students each are formed via group registration in TUWEL.
2. Upload the project plan (Deliverable 1) **to TUWEL by 12.04.2026, 23:55 at the latest.**
 - a. The project plan must be submitted in one of the following formats: .rtf, .doc, .pdf or .html! No Microsoft Project® or other project management tool-specific format is allowed for the project plan! Make sure that there are no links to external objects (images, diagrams) in the documents. Make sure that the documents are not exorbitantly large (e.g. by including very large images).
 - b. The file name for the project plan submission as pdf would therefore be **"DSE26Group<nn>-Projectplan.pdf"**
 - i. <nn> corresponds to their group number formatted to two digits
 - c. If you choose for example HTML as the format and the document contains several HTML files, pack the files into a ZIP archive. Make sure that the document can be browsed directly from the ZIP archive, i.e. the ZIP archive does not have to be unzipped to the hard drive.
 - i. The file name for the project plan submission as a zip archive is **therefore "DSE26Group<nn>Projectplan.zip"**
3. Book an appointment for your submission interview per group at TUWEL **by 31.05.2026, 23:55 at the latest.**
 1. The submission interviews will take place from **15.06.2026 to 19.06.2026.**
4. Upload the total submission of all deliverables in TUWEL **by 07.06.2026, 23:55 at the latest.**
 - d. All files of the total submission must be submitted in one of the following formats: .rtf, .doc, .pdf or .html! Make sure that there are no links to external objects (images, diagrams) in the documents. Make sure that the documents are not exorbitantly large (e.g. by including very large images).
 - e. **All** submitted documents must bear the number of the deliverable and the document title (as specified in the task) as file name or document title! The title must be visible on the first page and via footers throughout the document.
 - f. All files of the total submission must be stored in a meaningful directory structure that corresponds to the numbering of the deliverables.
 - g. Be sure to enclose Deliverable 1 (project plan) with the submission.
 - h. The file name of the submission must **be** DSE26Group<nn>-Submission.zip
5. The file name of the project plan file in the overall submission is therefore **"1_Projectplan.pdf"** or **"1_Projectplan.zip"** and the directory name in which the project plan is to be saved must be **"1_Projectplan"** – analogous to all other deliverables.
 1. The presence of all **group members is required** for the presentation within the submission interview.
 2. You present your solution using
 - Brief overview of the solution
 - Deployment of the solution into GCP in the sense of IaC (Infrastructure as Code) using Kubernetes manifests (YAML files)
 - The system must be deployed on GCP for presentation (no local installations)!
 - Live demo of the simulation scenario

- Insight into the running system (Logging, Google Cloud Console, error handling, etc.)
 - Code Walkthrough
 - Build run with successful execution of all tests
3. Each group member must present the parts she or he has worked on.
 4. Each group member must know the overall project - i.e. also the parts presented by the other group members - fully understand it and be able to answer questions about it.
 5. We reserve the right to award fewer points to group members if the impression arises that they have not adequately contributed to the overall scope of the exercise.

5.1 Important notes and tips

1. **Study the information carefully!** Deliverables are always forgotten, or topics are missing. To complete the exercise positively, all deliverables must be handed in. We reserve the right not to accept deliverables that do not formally comply.
2. Do not use **publicly accessible** groupware accounts or public source code repositories! Other groups could have access to your source code or get your documents!
 - a. For example, use Bitbucket (<https://bitbucket.org/>) – here you can set up free private workspaces.
 - b. GitHub now also offers private repositories for a maximum of 3 collaborators (<https://github.com/pricing>).
 - c. Gitlab (www.gitlab.com) is another possibility.
3. Assign roles or responsibilities within the group (e.g. for project management, coordination, technology, tools, documentation). Schedule project meetings even during the runtime, e.g. at certain milestones. Do not save everything for the last moment.
4. During the presentation, you must be informed about the overall project – not just about your own part!
5. If you make assumptions in the solution based on incomplete information, be sure to document them! Assumptions **must not** trivialize the solution.
6. Plagiarism: Please do not copy solutions from other groups or solutions offered on the Internet. If we detect plagiarism, we will award 0 points, and you will not be able to complete the exercise positively. Of course, it is allowed to discuss problems and possible solutions with your colleagues, but the solution submitted must be 100% created by you and unique. Even if groups work together, the solutions must not be too similar.
7. Deadline postponements: Deadline postponements are only granted in special situations after an individual agreement has been made with the course instructor. If you can't finish your example in time, turn in your example unfinished – that's still better than giving nothing at all. Be aware that something can always go wrong when handing in and plan accordingly. Submit your solution in good time and be aware that the servers may be overloaded shortly before the deadline. We cannot accept email submissions or other uploads. If you have problems with the submission, send a SHA-512 hash of your submission to the course instructor in time for the submission deadline. Using this hash, we can check whether the

submission has been modified after the deadline (after you have uploaded the submission after the deadline, as soon as the upload problem is fixed).

5.2 Important dates

Sun, 12.04.2026, 23:55: Deadline for submission of project plan via TUWEL

Sun, 31.05.2026, 23:55: Deadline for booking the Submission Interview

Sun, 07.06.2026, 23:55: Deadline for the total submission of all deliverables via TUWEL

Mon 15.06.2026 to Fri 19.06.2026: Submission interviews

Tue, 23.06.2026, 16:15-17:45: Exam

6 Summary of the exercise contents

1. Project planning
2. Architecture and design of a distributed system in the context of microservices, containers and orchestration mechanisms
3. Implementation
4. Test
5. Deployment
6. Creation of a maintenance manual
7. Project work: Group communication and cooperation
8. Presentation skills